场景营销前端 AI Coding — 从问题到方案

大淘宝技术 2026年6月22日 06:09 江苏 22人

本文深入分析AI编程（AICoding）效率瓶颈，指出核心问题在于大模型的注意力机制限制、上下文膨胀与注意力坍塌，以及人机协作模式不匹配；提出通过外置“DeepResearch”型 Agent 分离“上下文准备”与“编码执行”，以多模态输入、结构化任务、持久化分析和增量更新提升真实提效。

01 引言

2025 年，AI Coding 工具已经成为开发者的标配。Cursor/Qoder、Claude Code、Codex 等产品被广泛地使用，"AI 将取代程序员" 的论调此起彼伏。然而，在实际的业务开发中，我们发现一个尴尬的现实：**AI 生成了 80% 的代码，但开发效率并没有提升 80%**。

这篇文章试图回答 2 个问题：

1. **为什么 AI Coding 没有达到预期的效果？** —— 是工具不够好，还是我们对大模型的能力边界理解不足？
2. **如何真正提升 AI Coding 的效率？** —— 关键不在于"让 AI 写更多代码"，而在于"减少人工干预次数"

02 大模型的能力边界与上下文困境

在讨论如何用好 AI Coding 之前，我们需要先理解大模型的本质限制。这是为了找到大模型正确的使用方式。

> 本章的分析以说明核心概念为主，部分数字为示意性说明而非实测数据；模型能力仍在持续演进。但这些限制的方向性结论在当前实践中仍然成立，不影响后续章节的工程判断

注意力机制的计算瓶颈

Transformer 架构的核心是自注意力机制（Self-Attention），它需要计算每个 token 与所有其他 token 的关联度。这带来了一个根本性的限制：**计算复杂度和内存需求都是 $O(n^2)$**。

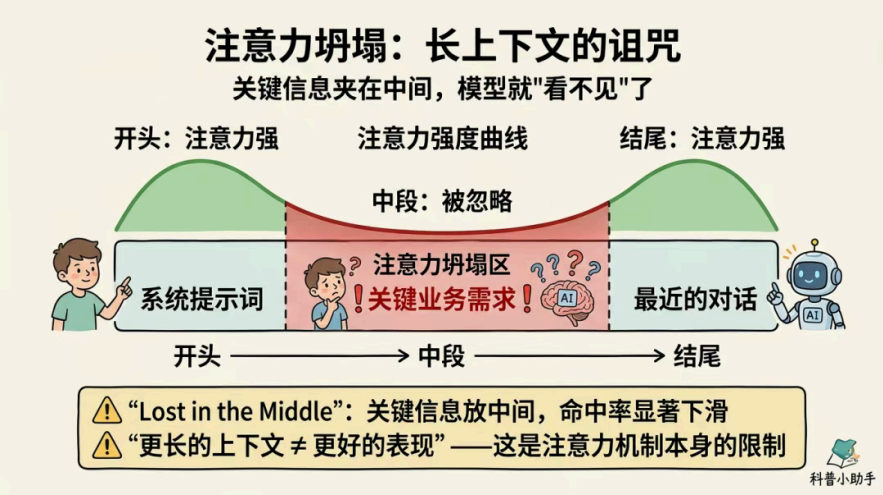
上下文长度	注意力矩阵大小	相对成本
4K tokens	16M	1x
32K tokens	1024M	64x
128K tokens	16384M	1024x
256K tokens	65536M	4096x

> 需要说明的是，实际工程中已有 Flash Attention、GQA、KV Cache 等优化手段大幅降低了这个开销，推理时每新增一个 token 的计算量是 O(n) 而非完整的 $O(n^2)$。但上下文越长、性能越容易下降这个方向性结论仍然成立——优化只是缓解，不是消除。

这解释了一个现象：模型厂商虽然宣称支持百万级上下文窗口，但实际使用中性能会显著下降。

用前端开发者熟悉的概念来理解：假设你写了一个页面，里面有 10 万个 DOM 元素，每个 DOM 都要和其他 99999 个做关联计算（比如"谁离我最近"、"谁和我相关"），这个计算量是灾难性的。

注意力坍塌：长上下文的诅咒



研究表明，当上下文增长时，会出现一种被称为"**注意力坍塌（Attention Collapse）**"的现象：模型的注意力过度集中在少数 token（通常是最近的内容），大量早期上下文处于"惰性状态"。

当上下文从几千个 token 扩展到几十 K 甚至更长时，多项实验都发现了一个共同现象：模型在中段位置查找信息的能力明显变弱，推理结果变得不稳定，幻觉概率也会上升。斯坦福的 "Lost in the Middle" 研究就是一个典型例子，关键信息夹在长上下文中间时，命中率会明显下滑。

更长的上下文 ≠ 更好的表现。这是注意力机制本身的限制，不是模型“还不够聪明”。

就像在一个 5000 行的文件里工作，光标在第 5000 行时，你很难准确回忆起第 10 行的变量名是什么。大模型也一样。

编码场景的上下文困境

在典型的 AI Coding 场景中，上下文的构成大致如下：

组成部分	Token 数量
系统提示词（规则、约束）	5-10K
代码库上下文	20-40K
需求文档	15K
历史对话	30K
工具定义（MCP、Skills）	10-40K
中间状态（分析过程）	10-20K
总计	**100-150K**

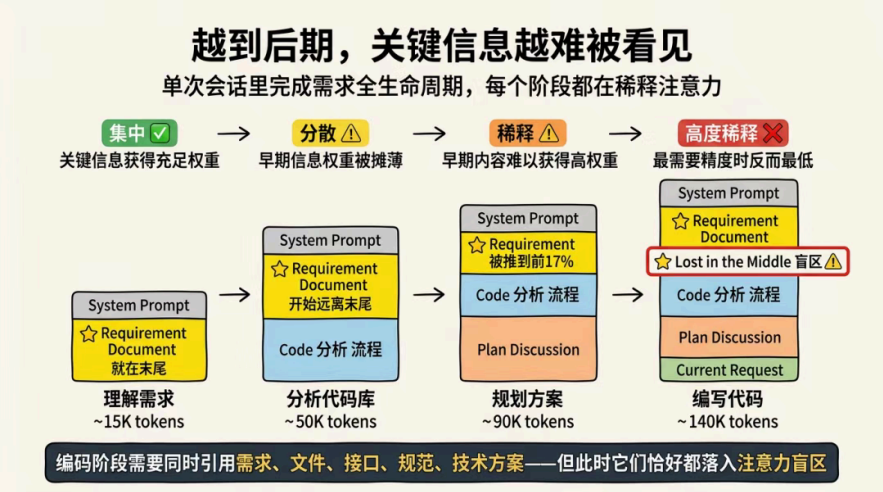
这里存在三个核心问题：

- 问题一：信号被噪声淹没

编码实际需要的信息只是：一段功能描述、2-3 个相关文件、几条规范。但模型实际接收到的是 100K+ tokens 的混合上下文。

> 用 CSS 的概念来理解：真正需要的规则（信号）是 `.Button-primary { color: blue; }`，但同时存在大量低优先级规则（噪声），比如系统提示词像 `* { color: red !important; }`、代码库上下文像 `div.container ul li a span { ... }`、历史对话像 `@media (max-width: 1440px) { ... }`。关键信号被噪声淹没，模型"看花了眼"。

- 问题二：注意力持续稀释

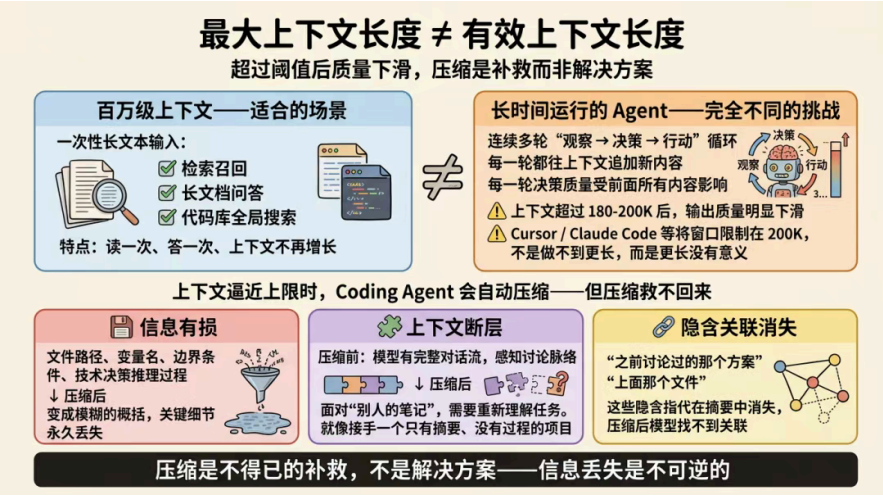


但当我们在一个会话里完成需求的完整生命周期时，每个阶段的分析过程对当时有用，完成后却变成下一阶段的背景，和真正需要的信息一起争抢有限的注意力权重：

阶段	上下文构成	规模	关键信息的处境	注意力状态
理解需求	系统提示词 + 需求文档	~15K	就在末尾，上下文小	集中，关键信息获得充足权重
分析代码库	系统提示词 + 需求文档 + 代码分析过程	~50K	开始远离末尾，中间过程开始堆积	分散，早期信息的权重被摊薄
规划方案	系统提示词 + 需求文档 + 代码分析过程 + 规划讨论	~90K	被推到前 17%，被分析过程包围	稀释，早期内容难以获得高权重
编写代码	… + 需求文档 + 代码分析 + 规划讨论 + 当前请求	~140K	被埋在中间，四面被噪声包围	**高度稀释，最需要精度时反而最低**

编码需要同时引用散布在上下文各处的多种信息 —— 原始需求、待修改文件、接口约定、编码规范、前面确定的技术方案。但到了编码阶段，140K token 争抢有限的注意力权重，这些信息恰好都落入 1.2 节描述的 "Lost in the Middle "盲区：离末尾太远（注意力弱），离开头也不够近（被系统提示词占据），夹在大量中间过程产物之间。

- 问题三：有效上下文存在天花板



模型宣传的 "最大上下文长度" 和实际能保持高质量输出的 "有效上下文长度" 是两回事。虽然当前部分模型支持百万 token，但从实际使用来看，**上下文超过 180 – 200K 左右后，输出质量就开始明显下滑**。这也是为什么大部分 Coding Agent（Cursor、Claude Code 等）至今仍将上下文窗口限制在 200K 左右。

百万级上下文更适合一次性长文本输入的场景：检索召回、长文档问答、代码库全局搜索。但它不天然等于适合长时间运行的 Agent。Agent 不是只做一次性阅读和回答，**而是在连续多轮 "观察—决策—行动" 的循环中持续依赖上下文** — 每一轮都在往上下文里追加新内容，每一轮的决策质量都受前面所有累积内容的影响。这和"读一篇长文档然后回答问题"是完全不同的挑战。

另外，当 Agent 的上下文逼近上限时，Coding Agent 通常会自动对早期内容做摘要压缩——把完整对话压成几段总结。但实践中，**压缩后输出质量往往会急剧下降**，原因是多方面的：

- **信息有损**：摘要会丢失具体的文件路径、变量名、讨论过的边界条件、技术决策的推理过程。这些对编码至关重要的细节被压成了模糊的概括
- **上下文断层**：压缩前，模型有完整的对话流，能感知讨论的脉络。压缩后，面对的是一段没有参与过的" 别人的笔记"，需要重新建立对任务的理解
- **隐含关联消失**：对话中大量信息是隐含的——"之前讨论过的那个方案"、"上面的某个文件"。 摘要不会保留这些指代关系，压缩后模型会丢失这些关联

压缩本质上是一种不得已的补救，而不是解决方案。它试图在上下文膨胀到不可控之后做抢救，但信息丢失是不可逆的。

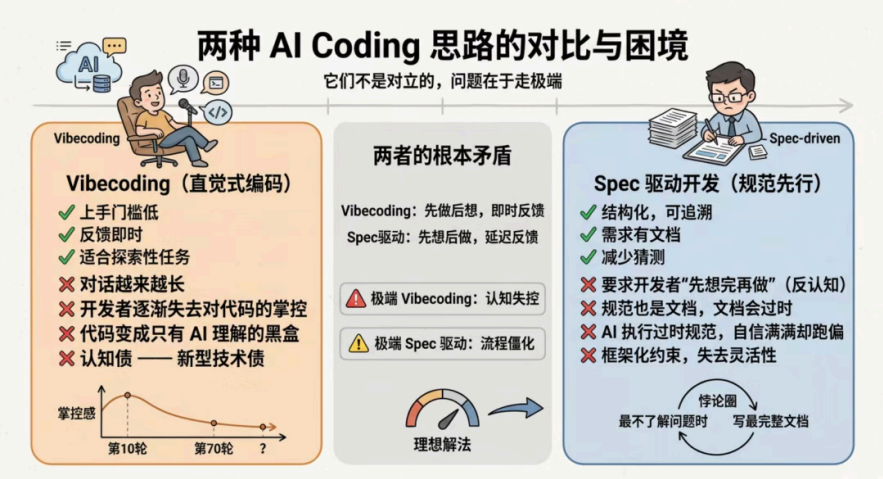
- 关键启示

理解这些限制后，我们可以得出几个重要启示：

1. **不要迷信"超长上下文"**：就像不会把整个 node_modules 塞进一个 bundle 文件，上下文要精准裁剪
2. **控制上下文规模是关键**：上下文越小，注意力越集中，每条信息获得的权重越高
3. **分阶段注入上下文**：理解需求时只给需求文档，编码时只给相关文件，避免一次性塞入过多 tokens

03 AI Coding 的尝试与问题

面对上一章提到的这些限制，业内目前形成了两种主流的应对思路。

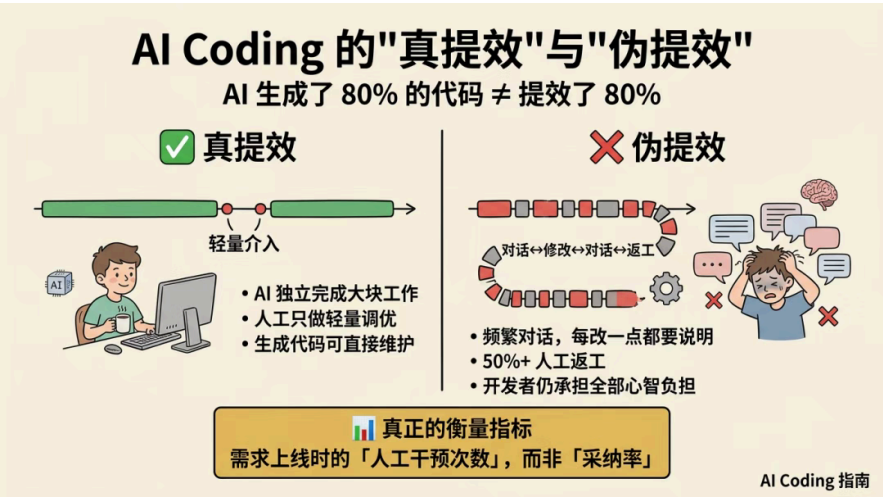


第一种是 **Vibecoding** — 直觉式编码。开发者和 AI 在 IDE 里高频对话，快速迭代，凭直觉推进。这种方式的优点很明显：上手门槛低、反馈即时，特别适合探索性的工作。很多人第一次体验到 AI Coding 的惊艳感，就是在这种模式下。

第二种是 **Spec 驱动开发** —— 先写清楚需求规范，再让 Agent 按规范执行。这个思路的出发点很好：与其让 AI 猜你要什么，不如把需求写明白。社区里 Speckit、OpenSpec 这些工具都是这个方向的尝试。它的价值在于让需求有迹可循，不会随着对话推进而失焦。

这两种思路看起来都有道理，实际落地时也各有各的优缺点。但在展开讨论之前，我们先看一组真实数据——在我们的项目实践中，AI Coding 到底带来了多少提效？

真提效 vs 伪提效



在实际业务开发中，我们统计了使用 AI Coding（类似 Speckit 的工程模式） 的效果，发现了一个值得深思的现象：

真提效的特征

- AI 能独立完成大块工作，开发者只需少量交互

- 人工只做轻量调优（参数、样式），不重写核心逻辑
- 生成的代码可维护，不会成为技术债

伪提效的特征

- 频繁交互对话，每改一点就要重新说明
- AI 生成后需要大量返工（50%+ 人工调整）
- 开发者仍需承担全部心智负担（架构设计、质量把控）

我们在一个真实项目中做了详细统计：

场景	表面“提效”	真实提效	AI 实际贡献	原因分析
浏览倒计时Hook	70%	真提效 70%	完整 Hook 实现	标准模式，一次生成
JS Error修复	80%	真提效 80%	快速定位问题	减少排查时间
订阅封装	70%	真提效 70%	完整封装逻辑	需求明确，标准实现
砸蛋明细	50%	真提效 50%	**Spec 转代码**	结构单一，微调样式
兑换模块	30%	负提效	生成垃圾代码	50% 返工，质量差
兑换联调	40%	0% 提效	充当打字员	频繁交互，低产出
砍价 Feeds	50%	实际 5-10%	改了按钮	核心靠复用，AI 改小功能
Feeds 复用	50%	存疑 30%	AI + 人工改造	不清楚交互成本
分享中间页	0%	0%	无	需要业务理解
版本升级	0%	0%	无	架构决策

结论：AI 只在约 **20%** 的标准化场景中真正有效，**80%** 的复杂场景效果差甚至反效果。

Spec 驱动开发的演进与困境

- 传统 Spec 模式的问题

早期的规范驱动开发理念很直接：**先写清楚需求文档，再让 AI 执行**。但在实践中遇到了根本性的挑战。

问题一：规范也是文档，文档会过时

社区里有一段文章说得很透彻：

> “你唯一能百分百信任的文档，就是代码本身。设计文档、更新日志、README、架构图，这些东西写完几乎立刻就过时了。让文档和不断变化的系统保持同步，需要持续投入成本，而这个工

> 作每次都会败下阵来。""规范也是文档。区别在于代价不同——过时的设计文档只会误导碰巧读到它的下一位工程师，而过时的规范会误导 Agent。它们会自信满满地执行一个早已脱离实际的计划，根本不会发现哪里不对。”

这段话指出了 Spec 驱动的核心矛盾：它依赖文档的准确性，但文档天然会过时。

问题二：认知模式不匹配

开发者的自然思维方式是：想法（模糊）→ 代码尝试 → 运行观察 → 调整想法 → 继续编码。这是一个**探索式、迭代式、具象化**的过程。开发者通过编写代码来澄清自己的想法。

但传统 Spec 模式要求的思维方式是：想法（模糊）→ 结构化分析 → 完整文档 → 交给 AI → 代码产出。这要求开发者在写代码之前，就完整地、抽象地、结构化地表达需求。

维度	探索式编码	传统 Spec 驱动
思维模式	具象思维（通过代码思考）	抽象思维（通过文字思考）
工作方式	边做边想	先想后做
反馈循环	即时（运行代码）	延迟（等 AI 实现）
心智负担	低（不需要完整规划）	高（需要预见所有细节）

传统 Spec 模式的悖论：它要求你在最不了解问题的时候（开始），做出最完整的描述。

- 大模型时代的新型 Spec 工具

社区意识到了这个问题，催生了一批新工具：**OpenSpec**、**Spec Kit**、**delta-spec** 等。它们的核心思路是：**让 AI 生成规范，而非人写规范**。

以 OpenSpec 为例，典型工作流是：

1. **Proposal（提案）**：用户只需简单描述（如"删除顶部导航的团队标签"），AI 生成结构化的变更提案
2. **Review（审核）**：人审核 AI 生成的规范，确认理解正确
3. **Implement（实现）**：AI 根据确认的规范执行编码
4. **Archive（归档）**：完成后将变更归档，自动更新系统规范

这种模式解决了两个问题：人不再需要从零写规范，规范随代码变更自动演进。

新型 Spec 工具仍存在的问题

然而，这些工具在复杂业务场景中仍有局限：

问题一：框架化编码，灵活性受限

这些工具作为 IDE 的内置辅助（通过 slash commands 或者 mcp 工作），整个编码过程被框架化：必须按 `proposal → apply → archive` 的流程执行。编码变成了"执行框架指令"，开发者失去了灵活性 —— 想中途 "先写一下试试" 反而会打破框架预期。

更重要的是，这些工具倾向于**一次性面面俱到**，希望在一个流程中把所有事情全部做完。但这与很多开发者 "小步快跑" 的习惯相悖 —— 先完成主体功能，再逐步补充细节。

在当前的业务组织形式下，需求频繁变更几乎是常态，vibecoding 的小步快跑本身有它的道理：先把能上线的部分尽快交付，再在后续版本持续补全。但前提是，团队不能因为小步快跑就放弃对整体需求的深度理解。更理想的状态是，在前期把需求吃透、规划好大致的边界和演进路线，然后让具体实现按小步快跑的节奏推进。

问题二：上下文注入不足

这些工具主要面向**代码库内部的重构或功能迭代**。当需求来源是外部的 PRD、设计稿、口头沟通时，工具缺乏将这些上下文结构化注入的能力。AI 生成的规范只能基于代码库已有信息，无法理解 "这个需求的业务背景是什么"。

问题三：任务粒度单一

OpenSpec 等工具的设计假设是：**一个提案 = 一个原子变更**。但真实业务需求往往是"开发一个完整功能"，需要拆分成多个相互依赖的任务，跨越数天甚至数周。这种多任务编排能力不在它们的设计范围内。

问题四：缺乏深度分析

这些工具的 Proposal 阶段相对轻量 —— AI 快速生成规范，人快速审核。但对于复杂需求，可能需要**多角色分析**（产品、设计、技术视角）、**代码库深度探索**（查找可复用代码）、**主动澄清**（向用户提问确认边界）。这种“DeepResearch”能力超出了它们的定位。

更可行的方向，是把"生成上下文"这件事从"实际编码"中抽离出来。先用外置的 Agent 或工具完成深度分析、任务拆分和上下文整理，再在 IDE 里做短上下文的编码执行。编码阶段不需要被流程框架强行约束，生成的任务上下文可以完全采纳、部分使用，或者在必要时直接丢弃——开发者始终保留决定权。这样既能享受深度分析带来的完整性，又不牺牲小步快跑的灵活开发体验。

Vibecoding 的优势与代价

前面提到，Vibecoding 的优点是上手快、反馈即时。它告诉我们一个重要的事实：**人机协作需要紧密的交互循环**，不能把 AI 当成一个"丢进去就不用管"的黑盒。

但轮次一长，一个被忽视的问题就会浮现：**人与 AI 的窗口长度不对称**。

AI 看到的内容包括：系统提示约 10K tokens、历史对话 1–50 轮约 120K tokens、当前任务约 5K tokens、代码上下文约 30K tokens，总计 165K tokens 全部在注意力范围内。而人看到的只是最近 3–5 轮对话（大约 5K tokens，而且记忆模糊）。

这导致什么问题？

1. **人跟不上 AI 的节奏**：第 70 轮对话时，开发者已经忘了第 10 轮发生了什么，但 AI 还在基于那个上下文做决策
2. **开发者逐渐失去对代码的掌控感**：从"理解每一行"到"大致知道在做什么"到"希望它能跑就行"
3. **代码仓库的腐化**：代码变成了只有 AI 能理解的东西——一种新型的技术债：**认知债**

案例：经过 80 轮对话实现了一个复杂功能。代码可以运行，测试通过。一周后需要修改一个小功能：

> 开发者：这个函数是做什么的？
> （翻聊天记录）找不到当时的讨论...
> （问 AI）AI 给出了一个解释，但不确定对不对
> （试着修改）改了，跑不了了
> （问 AI）AI 说要同时改另外三个地方
> （绝望）算了，重新让 AI 写一遍...

没有人真正理解这个系统。

这不是指 Vibecoding 不好， Vibecoding 更适合短平快的任务，不适合需要长期维护的复杂需求。**问题不在工具本身，在协作模式。**

多模态能力的缺失

还有一个更容易忽略的问题：**大部分 AI Coding 工具还没有充分利用多模态能力**。

大模型的图片识别能力已经很强了，但在实际的开发工具中，这个能力往往被浪费：

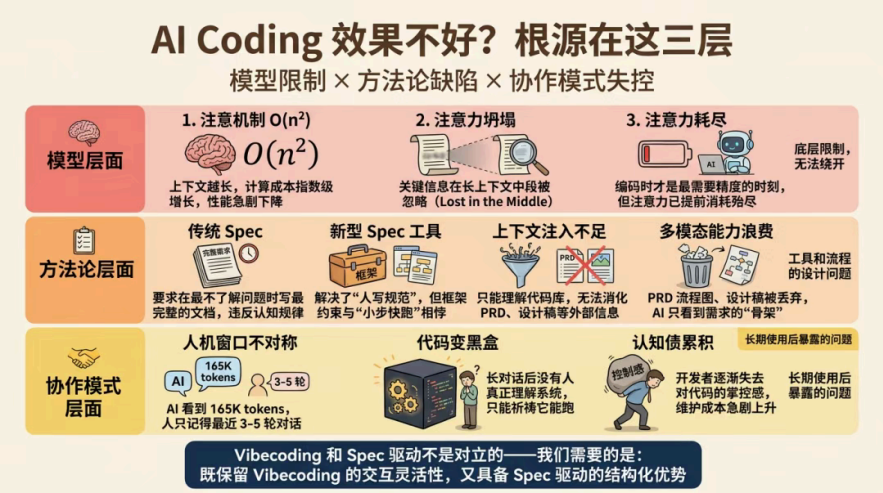
- **PRD 中的图片被丢弃**：产品经理用流程图说明状态流转、用时序图描述前后端配合、用视觉稿展示最终效果。这些图片往往比文字更精准，但下载 PRD 时只保留文本，生成方案时忽略这些图

- **设计稿只用来看，不用来理解**：UI 还原时，设计稿只是给开发者看的参考，AI 并不会真正"读懂"它。缺少对设计稿的结构化分析（组件层次、状态识别、可复用模式）

结果就是：AI 只看到了需求的"骨架"（文字描述），看不到"血肉"（视觉化的业务逻辑、交互细节、实际效果）。这解释了为什么 AI 生成的方案经常 "形似神不似"。

多模态不是锦上添花，而是补全信息链路的必要手段。

当前问题的根源总结



综合来看，当前 AI Coding 实践存在三层问题：

模型层面

- 注意力机制的 O(n²) 限制
- 上下文膨胀导致的性能下降
- 注意力坍塌不可避免

方法论层面

- 传统 Spec 模式要求的认知能力与人的自然思维方式不匹配
- 新型 Spec 工具（OpenSpec 等）解决了 "人写规范" 的问题，但引入了框架化约束，与"小步快跑"的习惯相悖
- 这些工具缺乏外部上下文注入和深度分析能力，任务粒度单一
- 多模态能力没有被充分利用，图片、视觉稿等关键信息被丢弃

协作模式层面

- Vibecoding 导致人机窗口不对称
- 长对话后代码变成黑盒

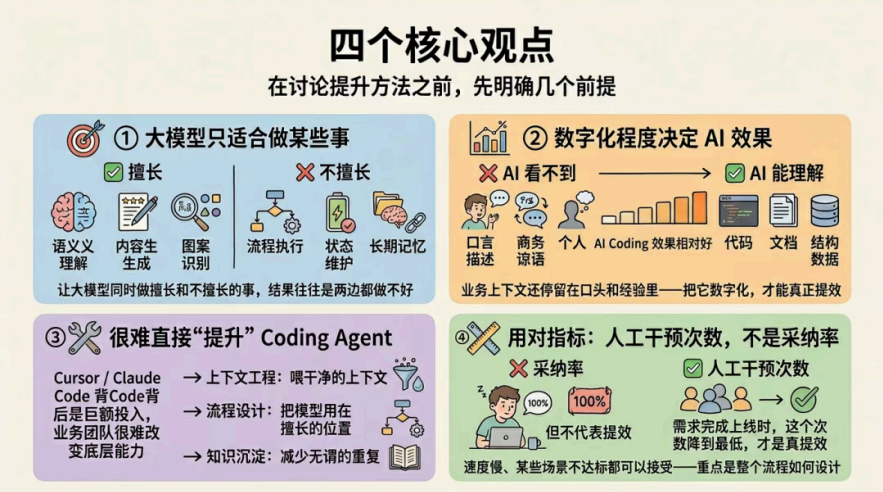
- 开发者失去对系统的理解

回过头看，Vibecoding 和 Spec 驱动其实不是对立的。Vibecoding 的价值在于即时反馈和探索效率；Spec 驱动的价值在于结构化和可追溯。问题在于，纯粹的 Vibecoding 会导致认知失控，纯粹的 Spec 驱动会导致流程僵化。

我们需要的是一种方式，既保留 Vibecoding 的交互灵活性，又具备 Spec 驱动的结构化优势。这也是我们尝试解决的问题。

04 如何提升 AI Coding

核心论点



在提出解决方案之前，我想先明确几个核心观点：

观点一：大模型只适合做某些事情

大模型的注意力机制和上下文困境，决定了它擅长的是**语义理解和生成**，而不是**流程执行和状态维护**。让大模型同时做它擅长和不擅长的事情，结果往往是两边都做不好。

观点二：数字化程度决定 AI 效果

一个领域的数字化程度越高，大模型能发挥的作用就越大。相比较其他很多 AI 落地场景，AI Coding 的效果相对要好一些，一个重要原因是：代码本身就是高度数字化的资产，在一个 repo 内，模型有足够的结构化上下文可以索引和理解。

但我们日常业务需求为什么效果不好？因为**除了代码外，还存在大量的业务上下文** —— 这些上下文停留在口头、业务黑话、经验里。我们应该做的是提升这部分的数字化程度，与 AI

Coding 建立链接。

观点三：我们很难直接“提升”Coding Agent 本身的能力

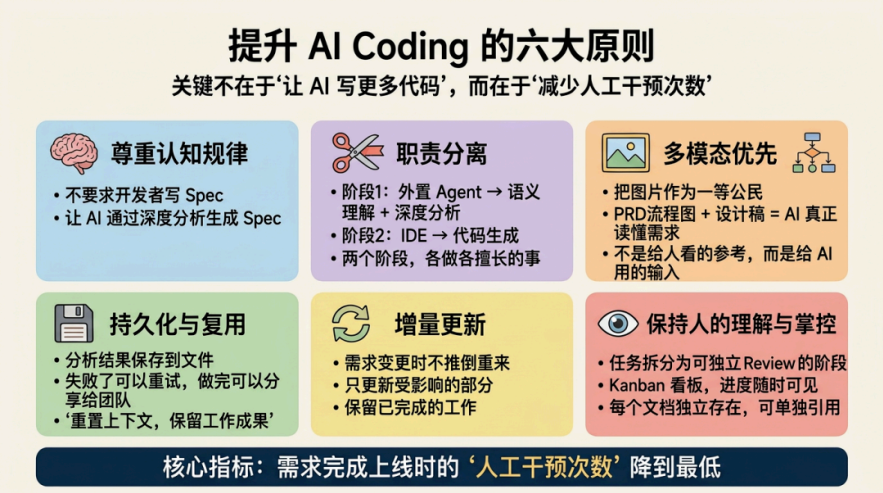
无论是基础模型，还是包了一层交互和工具编排的 Coding Agent，本质上都是由大模型厂和 IDE 厂主导演进的。基础模型、Cursor、Claude Code 这样的基建和工具背后是几亿、几十亿美金级别的投入/估值，靠业务团队很难真正改变它们的底层能力。我们能做的，是在上下文工程、流程设计和知识沉淀上做文章：**尽量把它们用在擅长的位置，喂干净的上下文，减少无谓的干扰。**

观点四：以"人工干预次数" 而非 "采纳率"为指标

AI Coding **不应该以采纳率为指标**。只要工程师坚持不写任何一行代码，完全通过对话的方式让模型输出，采纳率可以做到 100% —— 但这不代表提效。

真正的指标应该是：**需求完成上线时的人工干预次数**。只有这个次数降到最低，才能证明大模型真正提效。在这个目标的基础上，模型运行速度慢、某些场景能力不达标都是可以接受的——重点是整个流程如何设计。

解决方向

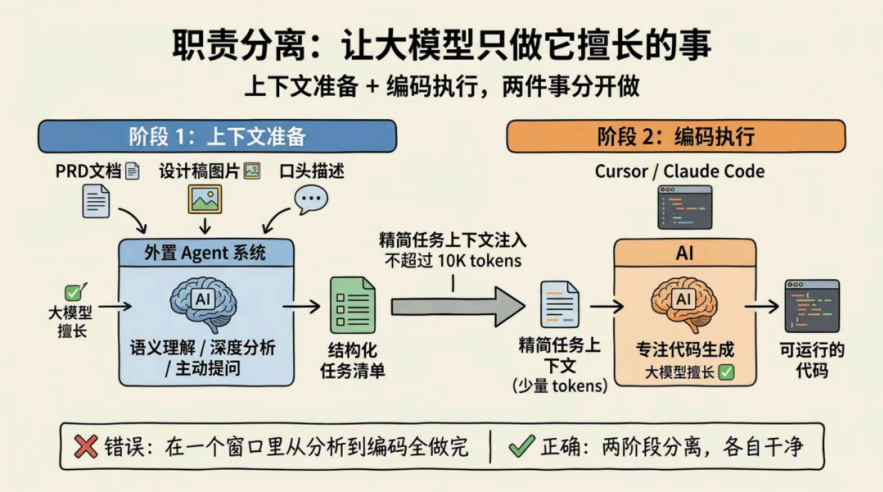


基于上述分析，我们提炼出几个核心原则：

原则一：尊重认知规律

不要求开发者写 Spec，而是让 AI 帮助他们**生成** "Spec"。错误方向是要求开发者写文档然后 AI 执行，正确方向是 AI 通过深度分析结合用户意图提取需求，生成文档，然后 AI 执行。

原则二：职责分离



将 "上下文准备" 和" 编码" 分离，让大模型在不同阶段做它擅长的事。阶段 1 是上下文准备：使用外置 Agent 系统，任务是对话提取需求、多角色分析、生成清晰任务，模型专注于语义理解（擅长）。阶段 2 是编码：使用 Cursor / Qoder / Claude Code，任务是基于清晰任务编写代码，模型专注于代码生成（擅长）。

原则三：多模态优先

把图片作为一等公民。PRD 里的流程图、设计稿、视觉效果图都要让 AI 真正读懂，做结构化分析。不是给人看的参考，而是给 AI 用的输入。

原则四：持久化与复用

深度分析的结果要持久化保存，支持重试和团队复用。分析一次的成果可以用多次，失败了可以基于已有分析重试，做完了可以分享给团队。

> 这也解释了社区里讨论的一个有趣的现象：同一个任务，在一个窗口对话几十轮后效果变差，但如果用同样的提示词新开一个窗口接着继续跑，反而效果不错。原因就是长对话导致的上下文膨胀和注意力下降。持久化的分析结果让你可以"重置"Agent 的上下文窗口，同时保留已有的工作成果。

原则五：增量更新

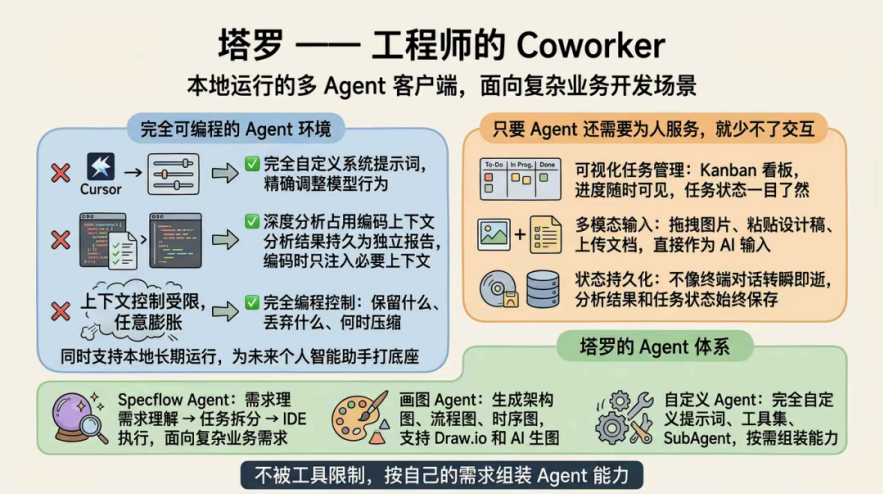
需求变更时不推倒重来，保留已有工作，只更新受影响的部分，这样才能应对真实业务中频繁的需求调整。

原则六：保持人的理解和掌控

部分 AI Coding 的问题是"单窗口端到端" —— 从分析到编码在一个对话窗口内连续完成，缺乏 Review 断点。更合理的做法是将任务拆分为可独立 Review 的阶段产出物：用 Kanban 或类似机制让任务进度透明、随时可介入；每个任务的相关信息都以附件形式独立存在，方便单独 Review；通过所见即所得的预览和编辑支持及时修正；每个文档（PRD 分析、技术方案、任务列表）都可独立引用和使用，不依赖对话上下文。

05 塔罗客户端与 Specflow Agent

定位：工程师的 Coworker



塔罗是一个基于 Claude Agent SDK 的本地运行的多 Agent 客户端，定位是**工程师的 Coworker**。

对比 Anthropic 的 Claude Cowork：一个 AI Native 的桌面 Agent 环境，能够自主访问文件系统、协调复杂任务、处理长时间运行的工作流。

核心理念：未来不再存在单纯的 Chat Bot，哪怕只是用来对话，也应当以 Agent 的方式设计。

为什么要做独立客户端？

坦白说，做客户端本身不是目的。我们真正需要的是一个**完全可编程的 Agent 环境** —— 能自定义提示词、控制上下文、实现工程兜底。如果现有的工具足够开放、支持完全定制，那有没有独立客户端其实问题不大。

但独立客户端在当下有一个额外的价值：2025–2026 年，各家都在布局 AI Native Agent。有了独立端，意味着未来可以让每个人的电脑上都部署一个长期运行的智能助手，类似 OpenClaw 那样的个人 Agent。这不是现在要做的事，但有了这个底座，后面会更好办。

为什么需要 GUI?

有一种观点认为 CLI 更高效、更适合自动化。但我的判断是：**只要 Agent 还需要为人服务，就少不了交互和确认。**

无论是 CLI 还是 GUI，交互是少不了的。而 GUI 提供了更好的：

- 可视化管理（任务看板、进度追踪）
- 多模态输入（拖拽图片、文件）
- 状态持久化（不像终端对话那样转瞬即逝）

塔罗的 Agent 体系

塔罗是一个**通用的 Agent 平台**。我们做的是系统性的事情：提供一套完整的 Agent 开发和运行环境。

目前内置的 Agent 包括：

- **Specflow**：需求理解 → 任务拆分 → IDE 执行，面向复杂业务需求
- **画图 Agent**：生成架构图、流程图、时序图，支持 Draw.io 和 AI 生图

同时支持**自定义 Agent**：开发者可以完全自定义提示词、工具集、SubAgent 配置。比如你可以创建一个专门写技术文档的 Agent。每个 Agent 都运行在独立的上下文中，互不干扰。

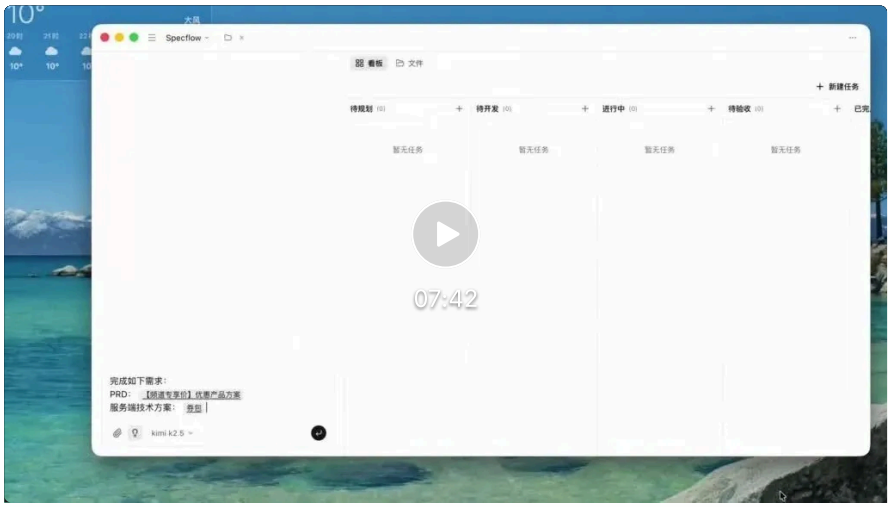
这套体系的价值在于：**你可以按自己的需求组装 Agent 能力，而不是被工具限制**。Specflow 只是其中一个面向 AI Coding 场景的实现。



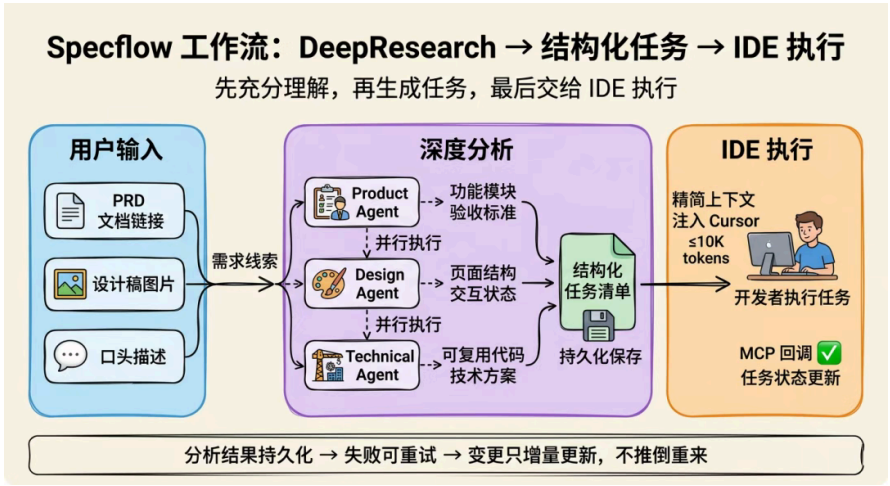
Specflow 的核心思路：“DeepResearch”

关于 Specflow 的命名：Specflow 本质上是**任务驱动开发**（Task-Driven Development），而非传统意义上的规范驱动。之所以保留 "Spec" 这个词，是因为它确实会生成结构化的任务规格，但这个规格是 **AI 深度分析后产出的**。它与第二章讨论的 OpenSpec 等工具有根本性的差异：外置而非内嵌、可选而非强制、深度分析而非快速生成。我没有找到更准确的命名，暂且沿用 Specflow —— 如果有更好的建议，欢迎交流。

有了可编程的 Agent 环境，我们就可以按自己的方式组织"上下文准备"这件事。Specflow 是塔罗里负责这个阶段的核心模块。



它的思路很简单：**先充分理解，再生成任务**。用户只需提供线索（PRD 链接、设计稿、口头描述），AI 深度探索（读 PRD、分析设计稿、探索代码库），进行多轮澄清（主动提问、确认理解），最后生成结构化任务。这本质上是 **DeepResearch** 的思路应用在开发场景。



为什么叫“DeepResearch”？因为在生成任务之前，AI 需要做大量的**主动探索**工作：

- **PRD 深度分析**
  - 自动读取 PRD 文档内容
  - 识别功能模块和业务流程
  - 区分前端任务和后端依赖
  - 提取验收标准和约束条件
- **设计稿图片深度分析**

- 识别页面结构和组件层次
- 分析交互状态
- 标注需要特殊处理的视觉元素

- **代码库深度探索**
  - 查找相似功能的现有实现
  - 识别可复用的组件、Hook、工具函数
  - 分析项目的技术栈和代码规范
  - 检测潜在的技术风险
- **多轮澄清**
  - 主动提问：哪些信息不明确？
  - 确认理解：这样理解对吗？
  - 边界确认：这个情况要处理吗？

这个过程类似于一个成熟的工程师拿到需求后做的事情 ——— 没有上来就写代码，而是先 **彻底搞清楚要做什么**。

PRD 理解与模块拆分

- PRD 分析的核心问题

给 AI 一个 PRD 链接，它能做什么？最简单的方式是 AI 读取 PRD 内容后说"好的，我理解了，这是一个砸金蛋活动..."——然后呢？没了。

这种"理解"是没用的，真正有用的 PRD 分析需要输出**可执行的任务清单**。

- Specflow 的 PRD 分析流程

PRD 分析分为四个步骤。

1. 第一步是文档读取与结构化：自动抓取 PRD 文档内容（支持钉钉文档、Aone等），识别文档结构（章节、表格、流程图），提取关键信息（功能点、业务规则、验收标准）。
2. 第二步是功能模块识别：按页面/功能拆分模块，识别模块之间的依赖关系，标注每个模块的复杂度。
3. 第三步是前后端职责划分：区分哪些是前端独立完成的、哪些依赖后端接口、哪些需要前后端联调，以及接口定义是否已确认。

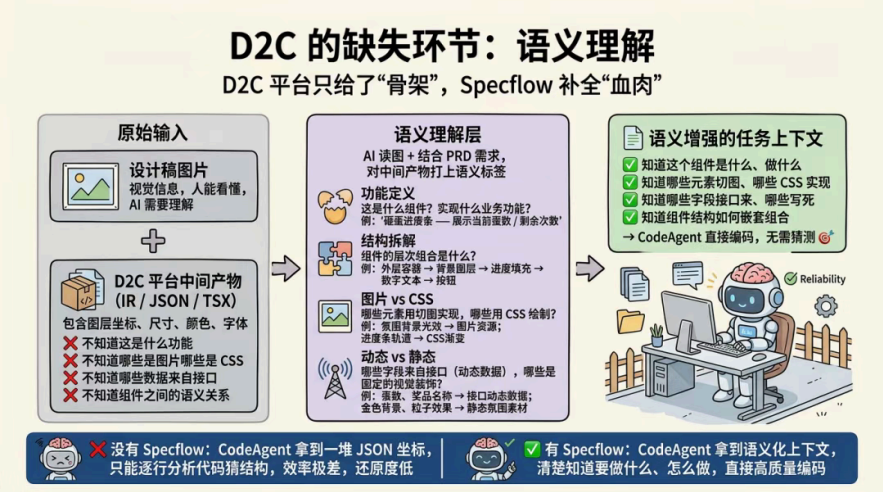
4. 第四步是任务生成：每个模块生成一个或多个任务，任务包含描述、验收标准、相关文件、依赖关系，任务按依赖关系排序。

- 模块拆分的原则

错误的拆分方式是按工序拆分，比如：任务 1 定义 TypeScript 类型、任务 2 编写组件结构、任务 3 实现样式、任务 4 添加交互逻辑、任务 5 接入接口。正确的拆分方式是按功能模块拆分，比如：任务 1 实现砸蛋主页面（包含完整的类型、组件、样式、交互）、任务 2 实现奖品明细浮层、任务 3 实现兑换流程。

为什么？因为**最终要交给 IDE 执行**。一个任务需要包含足够完整的上下文，让 Cursor 能够独立完成。按工序拆分会导致每个任务的上下文不完整，执行效果差。

D2C：为什么视觉还原度差？如何做到精确？



D2C（Design to Code）是前端开发中使用 AI 最多的场景，也是问题最多的场景。

首先要澄清一点：**前端没有独立的 D2C 任务**。"设计稿转代码" **只是一个独立环节**，在真实的前端开发中，**UI 还原只是功能开发的一部分**。一个完整的前端任务通常包括 UI 还原、交互逻辑、状态管理、接口联调 — 这些是一体的，不能割裂。单独跑一遍 D2C 生成的代码，往往只是个空壳，离 "能用" 还有很大距离。

- 为什么 D2C 还原度差？

信息丢失

设计稿是视觉信息，转成代码需要：

- 理解布局结构（Flex？Grid？绝对定位？）
- 识别交互状态（这个按钮按下去是什么效果？）
- 理解设计意图（这里为什么要这样排版？）

缺乏项目上下文

AI 生成的代码通常是"通用的"：

- 不知道项目用的是什么组件库
- 不知道项目的样式规范（rpx？rem？px？）
- 不知道有哪些可复用的组件

IR（中间表示）质量问题

很多 D2C 工具会先把设计稿转成 IR（JSON 描述），再生成代码。 问题是：

- IR 太大（包含大量无用信息）
- IR 不准确（识别错了组件结构）
- IR 不规范（每次生成结果不一致）

- Specflow 的 D2C 辅助方案

塔罗视觉还原方案已完成更新，当前文章内容仍停留在初始版本。对于基于 IR 方案的 D2C 还原场景，本文仍可作为一定参考。

D2C 平台通常会提供两类产物：设计稿图层转换的 JSON（IR）和 AI 优化过的 TSX 代码。但这些产物的问题在于**没有结合项目代码库和具体需求**，生成的内容无法直接使用。

D2C 平台输出的 IR 或 TSX，本质上是设计稿的结构化翻译：它知道这里有一个 120×40 的矩形、填充色是 `#FF6B00`、内部有一段文字 — 但它不知道这是一个"砸蛋按钮"，不知道按钮上的数字来自接口还是写死的，不知道背景的金色光晕是切图还是 CSS 渐变，也不知道这个组件在整个页面中扮演什么角色。

AI Coding Agent 拿到这样的中间产物，只能从坐标和颜色反推结构，相当于在没有标注的蓝图上施工 — 很容易建错。

Specflow 的做法是在 IR 之上叠加一层视觉语义分析：结合设计稿图片和 PRD 需求，对每个关键元素打上语义标签 — 这是什么、实现什么功能、是图片资源还是 CSS 实现、是接口动态数据还是静态氛围素材。有了这层标注，AI Coding Agent 不再需要猜测，拿到的是"读得懂"的上下文，直接进入高质量编码。

Specflow 的价值在于：**将 IR 与深度分析报告融合**。D2C 平台提供视觉层面的还原，Specflow 在前序阶段完成 PRD 分析、UI 分析、技术分析，产出了结构化的需求理解。在 Cursor 中使用的时候，将两者结合，才能生成真正符合业务需求的代码。

任务执行与复用

执行可以重试

AI 生成代码的成功率不是 100%。可能的问题包括：

- 理解偏差：AI 对需求的理解和预期不符
- 质量问题：生成的代码不符合规范
- 遗漏问题：漏掉了某些边界情况
- 执行失败：工具调用出错

错误的做法是

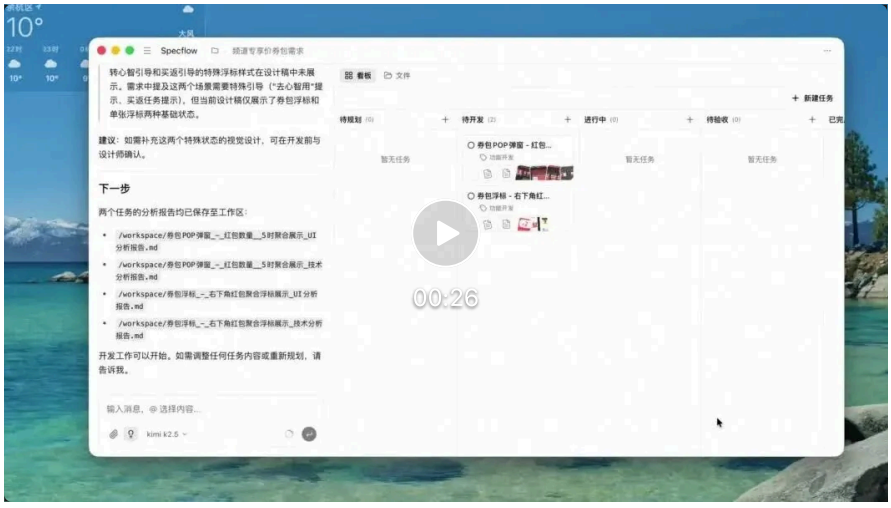
失败了就重新来，从头开始 —— 包括重新分析需求、重新理解代码库。

Specflow 的做法

前期深度分析的文档是持久化保存的。PRD 分析报告、UI 分析报告、技术方案报告，这些都独立存在于工作区。每次重试执行时，这些上下文可以直接复用，不需要重新分析。换句话说，重试的成本只是"再执行一次编码"，而不是"从头再来一遍"。

任务导入导出：团队复用

除了个人重试，Specflow 还支持任务的导入导出。一个需求分析完成后，可以导出为独立的任务包（包含 PRD 分析、UI 分析、技术方案、任务清单）。团队内其他人可以直接导入这个任务包，跳过分析阶段，直接开始执行。



这在几个场景下特别有用：

- **需求交接**：你分析完的需求，同事可以直接接手执行
- **经验沉淀**：好的分析结果可以作为团队资产保存

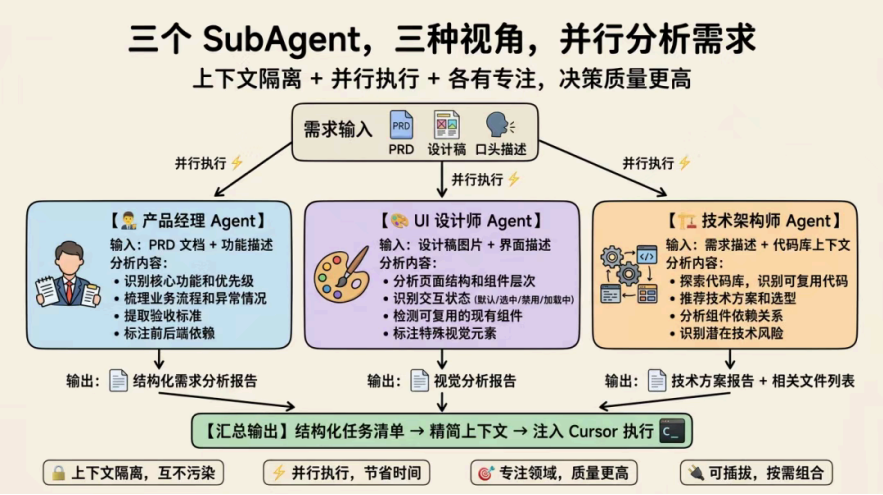
业务需求总是会变的。但是不能需求变了，之前的工作白费。

迭代更新机制

假设初始需求下任务状态是：任务 1 实现砸蛋主页面已完成、任务 2 实现奖品明细浮层已完成、任务 3 实现兑换流程正在进行中。这时需求变更了："奖品明细浮层要支持分享功能"。

Specflow 的处理方式是：首先识别变更影响范围（任务 2 奖品明细浮层），然后生成增量任务（任务 2.1 添加分享功能），保留已完成的工作（任务 1 不受影响），更新任务依赖（任务 3 可能需要感知分享状态）。而不是删除所有任务然后重新从 PRD 分析开始。

多角色 SubAgent 分析



Specflow 使用三个专业 SubAgent 从不同视角分析需求：

Product Manager（产品视角）

输入是 PRD 文档和功能描述。任务包括：识别核心功能和优先级、梳理业务流程和异常情况、提取验收标准、标注前后端依赖。输出是结构化的需求分析报告。

UI Inspector（设计视角）

输入是设计稿图片和界面描述。任务包括：分析页面结构和组件层次、识别交互状态和边界情况、检测可复用的现有组件、标注需要特殊处理的视觉元素。输出是视觉分析报告。

Tech Architect（技术视角）

输入是需求描述和代码库上下文。任务包括：探索现有代码库并识别可复用代码、推荐技术方案和技术选型、分析组件依赖关系、识别潜在技术风险。输出是技术方案报告和相关文件列表。

为什么要分成三个 SubAgent?

1. **上下文隔离**：每个 SubAgent 有独立的上下文窗口，不会互相污染。产品分析不需要知道代码细节。
2. **并行执行**：SubAgent 可以同时运行，缩短分析时间。
3. **专注性**：每个 SubAgent 只关注特定领域，提示词更精准，决策质量更高。
4. **可插拔**：不是每个任务都需要三个视角。D2C 任务可能需要视觉和技术分析，动效任务可能只需要技术分析。

提供给生码 IDE 的上下文

当 Specflow 完成分析，准备把任务交给 Cursor 执行时，会生成一个精简的任务上下文。这个上下文是结构化的 Markdown 文档，包含以下信息：

- **需求说明部分**：核心功能描述（比如展示砸蛋进度、触发砸蛋动画、展示奖品），以及验收标准（比如进度条准确反映当前蛋数、砸蛋动画流畅无卡顿、奖品展示支持多种类型）。
- **UI 分析报告**：完整的视觉描述（界面概况、空间布局、元素清单）、模块拆分方案、可复用组件模板识别、交互状态汇总、视觉资源清单（哪些需要切图、哪些可代码实现）。
- **技术方案报告**：系统概况（当前任务涉及的已有模块、新增/修改范围）、需求技术拆解、代码复用分析（可直接复用/需要扩展/可参考的组件和工具函数）、技术方案与选型、实现方案（整体架构、关键技术点思路、数据流和状态管理、文件清单）、风险与解决方案。
- **相关文件部分**：入口文件和组件目录的路径，标注哪些需要创建、哪些已存在。

这个上下文的特点：

1. **精简**：只有很少的启动 tokens，不会稀释 Cursor 的注意力
2. **结构化**：清晰的分类，Cursor 可以快速定位信息
3. **可执行**：包含具体的文件路径、代码引用，Cursor 可以直接使用
4. **自包含**：不需要额外的对话来澄清需求

可视化任务管理

Specflow 提供本地 Kanban 看板，按状态列展示所有任务：Backlog（待办）、Todo（计划中）、In Progress（进行中）、Review（审核中）、Done（已完成）。开发者可以直观地看到每个任务的状态。



为什么需要可视化管理？

1. **持久化保存**：不像 IDE 对话那样临时性，任务状态始终可见
2. **进度追踪**：随时了解开发进度，哪些完成了，哪些卡住了
3. **上下文恢复**：即使中断了开发，回来后可以快速恢复上下文
4. **需求变更追踪**：新增、修改、删除的任务都有记录

与 IDE 双向通信



Cursor Deeplinks

点击任务即可生成 Deeplinks，自动打开 Cursor 并加载相关文件和任务上下文。Deeplinks 格式包含任务上下文和相关文件路径，开发者直接开始编码，无需再次描述需求。

MCP 回调

整个流程是：Specflow 创建任务，通过 Deeplink 跳转到 Cursor，开发者在 Cursor 完成任务后，MCP 回调通知 Specflow（任务完成或需要修改），任务状态自动更新。

通过 MCP 协议，Specflow 和 Cursor 之间建立了双向通信：Specflow 向 Cursor 传递任务上下文，Cursor 向 Specflow 反馈执行结果。实现完整的任务管理闭环。

为什么不在 Cursor/Qoder 里完成这一切?

这是一个常被问到的问题。Cursor/Qoder 已经有了 Plan 功能，为什么还需要外置的 Specflow?

Cursor Plan 的定位

Cursor 的 Plan 模式是一个很好的功能：规划 → 执行。它适合**单任务、明确需求**的场景——你知道要做什么，只需要让 AI 帮你规划实现路径。比如"给这个函数加个缓存""重构这个组件，拆成三个子组件""实现一个防抖 Hook"。

但业务需求不是这样的

真实的业务需求通常是这样的："这个需求需要看下 PRD，做一个新的砸金蛋玩法，设计稿在 MasterGo 里，设计 xx 个页面，后端接口还没出，但字段大概就那些，还有，这个需求可能会跟之前的整点抢模块有复用..."

这里面有几个关键问题：需求是模糊的（PRD 链接、设计稿链接、口头描述混在一起）、上下文是分散的（代码、文档、设计稿分布在不同地方）、需求会变（今天确认的方案，明天可能就改了）、不是一次性任务（这是一个可能持续一两周的开发过程）。

Cursor/Qoder 内部做不到的事情

Cursor 是一个优秀的编码工具，但它的架构决定了有些事情很难做：

1. **提示词不可控**：Cursor 有自己的出厂设置提示词，这是它的底层约束，用户无法完全掌控。而独立 Agent（基于 Claude Agent SDK）可以完全自定义系统提示词，根据业务场

景精确调整模型行为。

2. **深度分析与编码职责冲突**：在 Cursor 里做深度 PRD 分析，分析过程会占用编码时的上下文窗口。分析产出的中间结果（PRD 理解、模块拆分、技术方案）无法结构化保存和复用，下次对话又要重新分析。外置系统可以把分析结果持久化为独立报告，编码时只注入必要的任务上下文。
3. **上下文控制受限**：在 Cursor 里，一旦对话开始，上下文就会持续膨胀。你无法精确控制哪些信息应该保留、哪些信息应该丢弃、什么时候压缩历史、如何保持关键信息的注意力权重。外置系统可以完全编程控制这些，Cursor 内部只能依赖它自己决定。
4. **工程兜底很难做**：Agent 运行是不稳定的。它可能跑偏了目标、遗漏了关键约束、生成了不符合规范的代码、一次没做好需要重试。在 Cursor 里，这些只能靠提示词和人工干预。在外置系统里，我们可以用在关键节点拦截和提醒、用状态机管理任务流程、持久化保存中间结果随时可以回溯。
5. **任务管理能力缺失**：一个需求可能拆成 5–10 个任务，开发周期跨越一周。Cursor 的对话没有任务状态管理、没有进度追踪、对话结束就丢失上下文、无法处理需求变更。

Specflow 的定位：Cursor Plan 的高阶版本

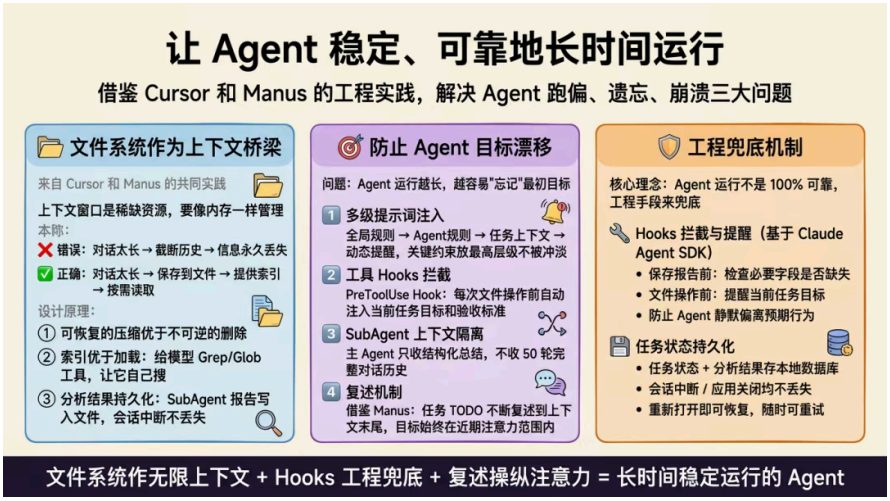
本质上，Specflow 做的事情和 Cursor Plan 是同一类：**规划 → 执行**。但它面向的是：

维度	Cursor Plan	Specflow
任务类型	单任务、明确需求	多任务、模糊需求
需求来源	用户直接描述	PRD、设计稿、口头沟通
执行周期	一次对话	跨天/跨周
需求变更	重新开始	增量更新任务
上下文控制	依赖模型	完全编程控制
工程兜底	提示词	Hooks + 状态机 + 重试机制



Specflow Agent 设计的技术细节

前面讲了 Specflow 的核心机制和工作流程，这一章聚焦在技术实现层面：如何让 Agent 稳定、可靠地长时间运行。



业界实践参考

在设计塔罗的 Agent 架构之前，我们调研了业界的最新实践。

Cursor 的动态上下文发现

Cursor 团队有一个关键洞察：**随着模型能力提升，预先提供更少的细节，反而能让 Agent 更容易自主地按需提取相关上下文**。

他们选择用文件系统作为上下文管理的核心抽象：

- 长工具响应转化为文件
- 对话历史保存为可检索的文件
- Agent Skills 按需加载
- 终端输出自动同步到文件

核心思想是：**上下文窗口是稀缺资源，要像内存一样管理**。

Manus 的六大原则

Manus（现已被 Meta 收购）从数百万真实用户交互中提炼出几个关键原则：

1. **围绕 KV 缓存设计**：缓存命中率直接决定用户体验和运营成本
2. **遮蔽而非移除**：避免动态添加或移除工具，保持上下文稳定
3. **文件系统作为无限上下文**：任何不可逆的压缩都带有风险
4. **通过复述操纵注意力**：不断更新 todo.md，将全局计划复述到上下文末尾
5. **保留错误尝试**：让模型从错误中学习
6. **不要少样本学习**：上下文越单一，Agent 越脆弱

其中"文件系统作为无限上下文""通过复述操纵注意力""保留错误尝试"这几条，在塔罗的设计中有直接体现。

文件系统作为上下文桥梁

塔罗大量使用文件系统作为上下文传递的媒介，而不是完全依赖对话历史。

- 设计原则

1. **可恢复的压缩优于不可逆的删除**

   错误做法：对话太长 → 截断历史 → 信息丢失

   正确做法：对话太长 → 保存到文件 → 提供索引 → 按需读取

2. **索引优于加载**

   不是把所有可能相关的信息都塞进上下文，而是给模型"查找能力"：

   - 代码库：提供 Grep、Glob 工具，让它自己搜索
   - 用户输入：上传的图片、粘贴的文档先保存到本地，通过 Read 工具按需读取
   - 分析报告：写入工作区，通过文件路径引用

3. **分析结果持久化**

   SubAgent 的分析结果不是一次性的，而是写入文件持久保存：

```
.tarot/workspace/
  ├── prd-analysis.md       # PRD 分析报告
  ├── ui-analysis.md        # UI 分析报告
  ├── tech-analysis.md      # 技术分析报告
  └── tasks.json            # 任务清单（示例，实际存放数据库）
```

   这样即使会话中断，也不会丢失分析结果。

防止 Agent 目标漂移

Agent 运行时间越长，越容易"忘记"最初的目标。我们采用了几种方法来解决这个问题：

方法一：多级提示词注入

我们采用分层的提示词架构：最上层是全局规则（始终生效，优先级最高），其次是 Agent 专属规则（根据 Agent 类型），然后是任务上下文（根据当前任务），最后是动态提醒（根据工具调用情况注入）。关键规则放在全局层级，确保不会被后续对话"冲淡"。

方法二：工具 Hooks 拦截

利用 Claude Agent SDK 的 Hooks 机制，在关键时刻注入提醒。例如在每次文件操作前，通过 PreToolUse Hook 自动提醒当前任务目标和验收标准，确保操作与任务目标一致。

方法三：SubAgent 上下文隔离

每个 SubAgent 有独立的上下文窗口。当 SubAgent 完成任务后，主 Agent 只收到结构化的总结（比如"Tech Architect 分析完成：发现 3 个可复用组件，推荐使用 Zustand 管理状态，详细报告见某个文件"），而不是整个对话历史（比如 50 轮对话的完整内容）。

方法四：复述机制

借鉴 Manus"通过复述操纵注意力"的原则，通过待规划任务形成 TODO 列表，每个任务完成后自动提醒下一个任务。这样任务目标被不断"复述"到上下文末尾，始终保持在 Agent 的近期注意力范围内，避免在长时间运行中偏离方向。

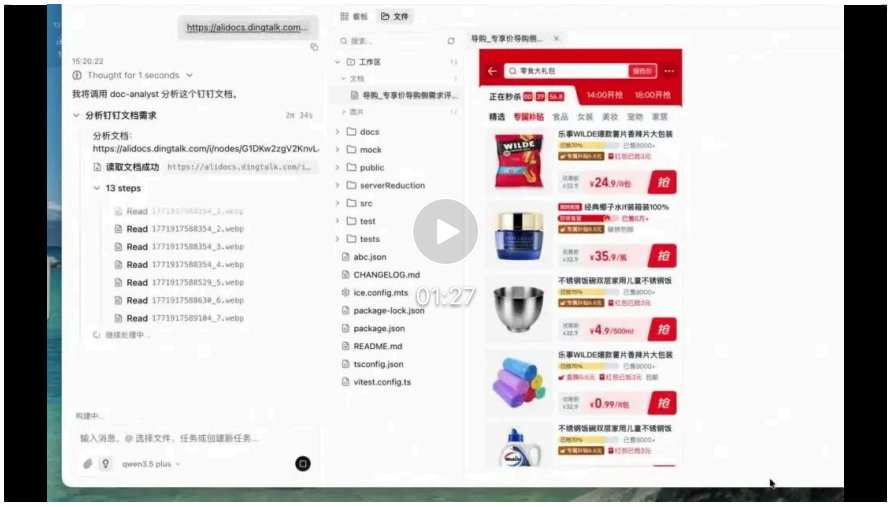
工程兜底机制

Agent 运行不是 100% 可靠的，需要工程手段来兜底：

工具 Hooks 拦截与提醒

利用 Claude Agent SDK 的 Hooks 机制，在关键节点做检查和提醒。比如在保存报告前，检查必要字段是否缺失；在文件操作前，提醒当前任务目标。这些机制确保 Agent 不会偏离预期。

下面演示一次需求清单生成的人工 Review 。



任务状态持久化

任务状态和分析结果都保存到本地数据库和文件系统。即使会话中断或应用关闭，重新打开时可以看到之前的任务和分析结果，不会丢失工作成果。

07 总结

核心观点回顾

1. **理解大模型的能力边界**：注意力机制的 O(n²) 限制、注意力坍塌、上下文困境，这些不是 bug，是 feature，理解它们才能正确使用 AI。
2. **区分真提效和伪提效**：以 "人工干预次数" 而非 "采纳率" 为指标。AI 生成了 80% 的代码不等于提效 80%。
3. **职责分离是关键**：将"上下文准备"和"编码"分离，让大模型在不同阶段做它擅长的事。
4. **DeepResearch 思路**：不要求开发者写 Spec，而是让 AI 深度探索、主动澄清，生成高质量的任务描述。
5. **外置系统的必要性**：Cursor 内部无法实现完整的工程兜底、上下文控制、任务管理。外置系统才能提供可编程的、可靠的流程控制。
6. **面向持续变更**：业务需求不是一次性的，而是持续变化的。任务管理需要支持增量更新、版本追踪。
7. **多模态是趋势**：大模型的图片识别能力越来越强，PRD 中的流程图、设计稿、视觉效果图都可以直接被 AI 理解。这让需求理解更完整、UI 还原更精确。

未来展望

有了 AI Native Agent 环境，未来可以做更多的事情：

1. **记忆加速迭代**：通过记忆系统加速 Agent“DeepResearch”的效率。第一次分析某类需求时，AI 需要完整探索代码库、学习项目规范、理解业务模式；这些分析结果持久化为记忆后，下一次遇到类似需求可以直接调用，跳过重复探索。越迭代越快。
2. **真正的自动化**：通过足够多的时间和尝试，Agent Teams 协作能完成绝大部分需求
3. **个人智能助手**：每个人的电脑上部署一个长期运行的 Agent，持续学习工作模式

AI Coding 的最终目标是**实现自动化**。但自动化不是一蹴而就的，它需要我们：

1. 理解大模型的能力边界
2. 提升业务上下文的数字化程度
3. 设计合理的人机协作流程
4. 不断积累和复用知识

这条路还很长，但方向是清晰的。

最后

说到底，我们做的事情没有什么黑科技，就是贴着两个朴素的原则：

1. **给模型足够的上下文**：让它看到完整的需求、设计、代码库信息
2. **减少对 Agent 的干扰**：编码时上下文要干净，不要塞太多无关信息

按这两个原则去做，怎么都会有效果。



本文作者木偶，来自淘天集团-用户场景营销技术团队。一支专注于探索AI等前沿技术与营销业务技术的融合，深度结合用户场景与营销业务的技术团队。依托大淘宝丰富的用户生态和多元化的消费场景，致力于通过技术创新提升用户体验，优化个性化营销能力，助力业务持续增长。通过AI驱动的精准推荐、场景化表达与动态策略调控，我们为用户创造更自然、更智能的购物旅程，为营销业务提供高效、敏捷的技术支撑，助力淘宝构建以用户为中心的全域营销技术体系。我们坚信技术是连接用户与价值的桥梁，持续探索创新边界，让营销更懂用户，用技术点亮每一个关键用户体验瞬间。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术
服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
967篇原创内容
公众号

阅读 4918

留言

写留言

还没有留言，来写首评